

Osaid Khan

+1 (945) 304-5781 | khanosaid726@gmail.com | www.linkedin.com/in/-osaid-khan/ | github.com/ozzyozbourne

EDUCATION

Pace University

Master of Science (MS) in Computer Science (GPA - 4.0/4.0)

New York City, NY

Jan 2024 - Dec 2025

Dr. A.P.J. Abdul Kalam Technical University

Bachelor of Technology (B.Tech) in Computer Science

Noida, India

Jul 2016 - Sep 2020

TECHNICAL SKILLS

Languages: Java, Python, Rust, Go, TypeScript/JavaScript, SQL

Distributed Systems & Design: Distributed control planes, Object Oriented Design, design patterns, multi-threaded and multi-tiered services, high availability, static stability, multi-tenant isolation, fail-over, query routing, reliability and scaling

AWS & Infrastructure: EC2, S3, Lambda, Step Functions, CloudWatch, CloudFormation, RDS, API Gateway, Serverless, Docker, Kubernetes, Temporal

Search, Data & Analytics: OpenSearch/Elasticsearch, full-text search, vector search, Pgvector, PostgreSQL, Redis, MongoDB, LlamaIndex, RAG retrieval

SDLC & Operations: CI/CD, code reviews, source control management, build processes, unit/integration testing, incident response, RCAs, OpenTelemetry observability, log analytics, GenAI coding assistants (Claude Code, OpenAI Codex)

PROFESSIONAL EXPERIENCE

Software Engineer

Sperse

Phoenix, AZ

Feb 2026 - Present

- Designed and operate a **distributed** orchestration **control plane** in **Python** and **Temporal** automating the full **lifecycle** of long-running workflows (**provisioning**, scale-out, retry, **fail-over**) with idempotent activities and **blast-radius**-scoped failure domains that hold **high availability** and **static stability** through partial node failure
- Built a **multi-tenant** CRM platform (**Python**, **LangGraph**, **FastAPI**) serving **10,000+** users, **routing** requests across 40+ sub-agents with tenant **isolation** and JWT-scoped authorization designed around real **failure modes**
- Engineered a **production MCP** server (**FastMCP**) exposing **160+** backend actions, using **Object Oriented Design** and **design patterns** to consolidate duplicate abstractions into one validated tool layer with retries and **metrics**
- Own **operations** end-to-end — **OpenTelemetry** traces, structured **log analytics**, and latency/error dashboards drive **incident response** and **RCAs**, converting recurring pages into automated **operational tooling**
- Apply **AI coding assistants** and **GenAI** tools (**Claude Code**, **OpenAI Codex**) across the **SDLC** — design, **testing**, **code review**, debugging — triaging 25-30 PRs/week through **CI/CD** while validating output for correctness and security

Software Engineer

Qualitest

Noida, India

Feb 2021 - Dec 2023

- Designed a cloud-agnostic Storage Service in **Java** and **Spring Boot** with **PostgreSQL** and **Redis**, using provider-abstraction **design patterns** to serve uploads, downloads, and deletes across **Google Cloud Storage**, **S3**, and **MinIO** with **durability** and **fail-over**
- Orchestrated **large-scale** long-running jobs with **AWS Step Functions**, cutting **AWS Lambda** timeouts and hardening error recovery via bounded retries and dead-letter replay across **100K+** weekly executions
- Reduced query latency and improved **scaling** of a **multi-tiered** analytics tier by tuning **PostgreSQL** with window functions, materialized views, and composite indexes, surfaced via **React** visualizations for **cross-functional** stakeholders
- Operated the service in **production** on-call — owned **CI/CD**, **code reviews**, **source control management**, and **build processes**, driving **RCAs** that removed dead code and configuration sprawl

PROJECTS

Dark Factory - distributed agent orchestrator | *Elixir, Rust, TypeScript*

github.com/ozzyozbourne/dark-factory

- Built a **distributed** orchestrator decomposing large coding requests into dependency-aware task DAGs scheduled across a **fleet** of multi-machine agents with heartbeat-based node failure detection and retries
- Implemented **multi-threaded** worker supervision with capacity-aware scheduling, isolating each task in its own failure domain so a crashed node is detected and rescheduled in seconds without losing DAG progress